```
import os, glob, json

os.environ.setdefault("AMAZON_ROOT", "/mnt/c/Users/legen/amazon_project")
os.environ.setdefault("AMAZON_RAW", "/mnt/c/Users/legen/amazon_data")

root = os.environ.get("AMAZON_ROOT", "/mnt/c/Users/legen/amazon_project")
raw = os.environ.get("AMAZON_RAW", "/mnt/c/Users/legen/amazon_data")
os.makedirs(root, exist_ok=True)
os.makedirs(raw, exist_ok=True)
os.makedirs(f"{root}/results", exist_ok=True)
os.makedirs(f"{root}/figs", exist_ok=True)
os.makedirs(f"{root}/cache", exist_ok=True)
os.makedirs(f"{root}/runs", exist_ok=True)
```

```
import sys
import importlib.util
import subprocess

_pipPkgs = [
    ("torch", "torch"),
    ("torchvision", "torchvision"),
    ("cv2", "opencv-python-headless"),
    ("pandas", "pandas"),
    ("sklearn", "scikit-learn"),
    ("matplotlib", "matplotlib"),
    ("tifffile", "tifffile"),
]
missing = [pip for mod, pip in _pipPkgs if importlib.util.find_spec(mod) is None]
if missing:
    print("Installing into active kernel:", " ".join(sorted(set(missing))))
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", *sorted(set(missing))])
else:
    print("Core packages already available in this kernel.")
```

```
Core packages already available in this kernel.
```

```
jpgsQuick = glob.glob(f"{raw}/**/train-jpg/*.jpg", recursive=True)
if not jpgsQuick:
    jpgsQuick = glob.glob(f"{raw}/**/*.jpg", recursive=True)
if len(jpgsQuick) > 1000:
    print(f"Local JPG dataset detected ({len(jpgsQuick)} files) — skipping any remote/Kaggle download
else:
    print("No large local JPG set found. Copy Kaggle train-jpg + train_v2.csv into AMAZON_RAW, then r
```

```
Local JPG dataset detected (40479 files) — skipping any remote/Kaggle download.
```

```
tifs = glob.glob(f"{raw}/**/*.tif", recursive=True)
jpgs = glob.glob(f"{raw}/**/train-jpg/*.jpg", recursive=True)
if not jpgs:
    jpgs = glob.glob(f"{raw}/**/*.jpg", recursive=True)

csvCandidates = (
    glob.glob(f"{raw}/**/train_classes.csv", recursive=True)
    + glob.glob(f"{raw}/**/train_v2.csv", recursive=True)
    + glob.glob(f"{raw}/**/labels_train_v2.csv", recursive=True)
)
```

```
if len(tifs) > 1000 and csvCandidates:
    paths = {
        "tif_dir": os.path.dirname(tifs[0]),
        "csv": csvCandidates[0],
        "image_ext": ".tif",
    }
    print("TIF files found:", len(tifs))
elif len(jpgs) > 1000 and csvCandidates:
    jpgDir = os.path.dirname(jpgs[0])
    csvNear = os.path.join(os.path.dirname(jpgDir), "train_classes.csv")
    csvPath = csvNear if os.path.exists(csvNear) else csvCandidates[0]
    paths = {
        "tif_dir": jpgDir,
        "csv": csvPath,
        "image_ext": ".jpg",
    }
    print("JPG files found:", len(jpgs), "(TIF torrent unavailable — RGB-only pipeline)")
else:
    raise SystemExit(
        f"Setup incomplete: tifs={len(tifs)} jpgs={len(jpgs)} csvs={len(csvCandidates)}. "
        f"Place data under AMAZON_RAW ({raw}) or set AMAZON_RAW / AMAZON_ROOT and re-run from the top

with open(f"{root}/paths.json", "w") as f:
    json.dump(paths, f, indent=2)
```

```
JPG files found: 40479 (TIF torrent unavailable — RGB-only pipeline)
```

```
import numpy as np
import pandas as pd
import cv2
import matplotlib
import matplotlib.pyplot as plt
%matplotlib inline
from concurrent.futures import ThreadPoolExecutor
from sklearn.model_selection import train_test_split

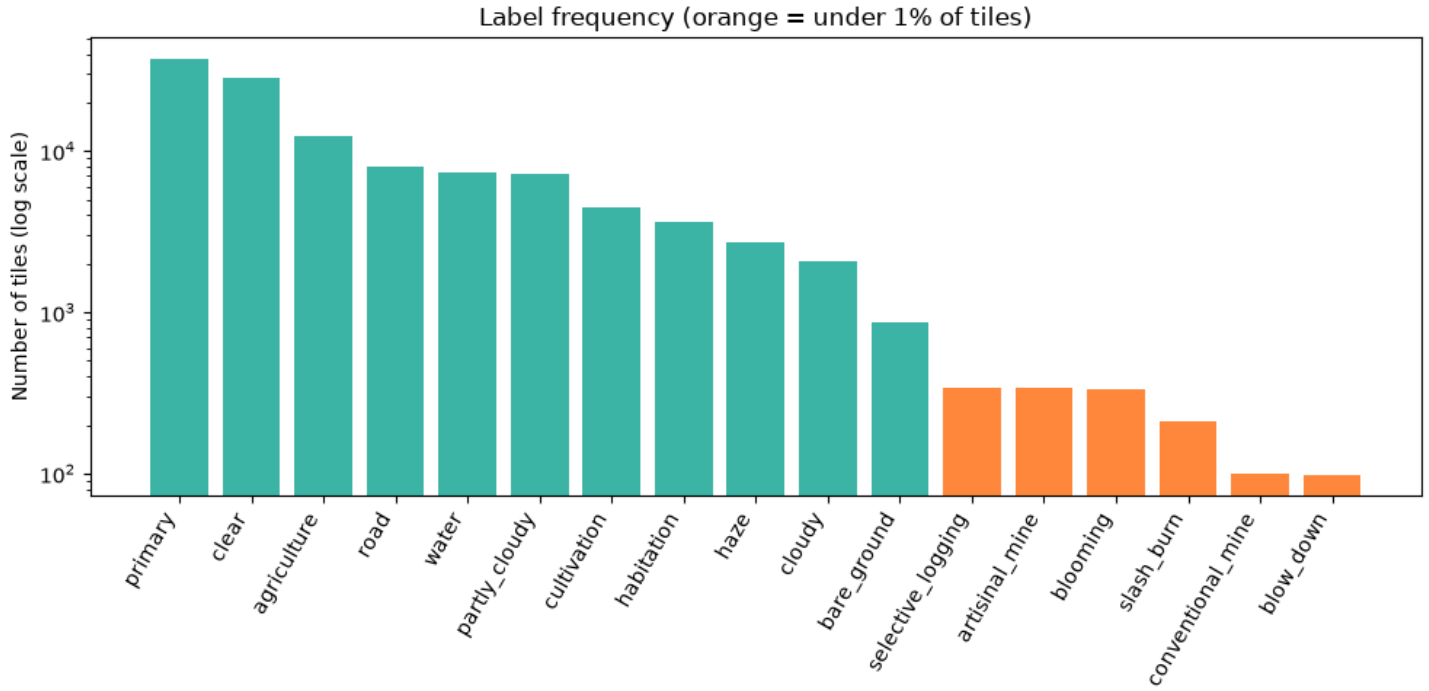
root = os.environ.get("AMAZON_ROOT", "/mnt/c/Users/legen/amazon_project")

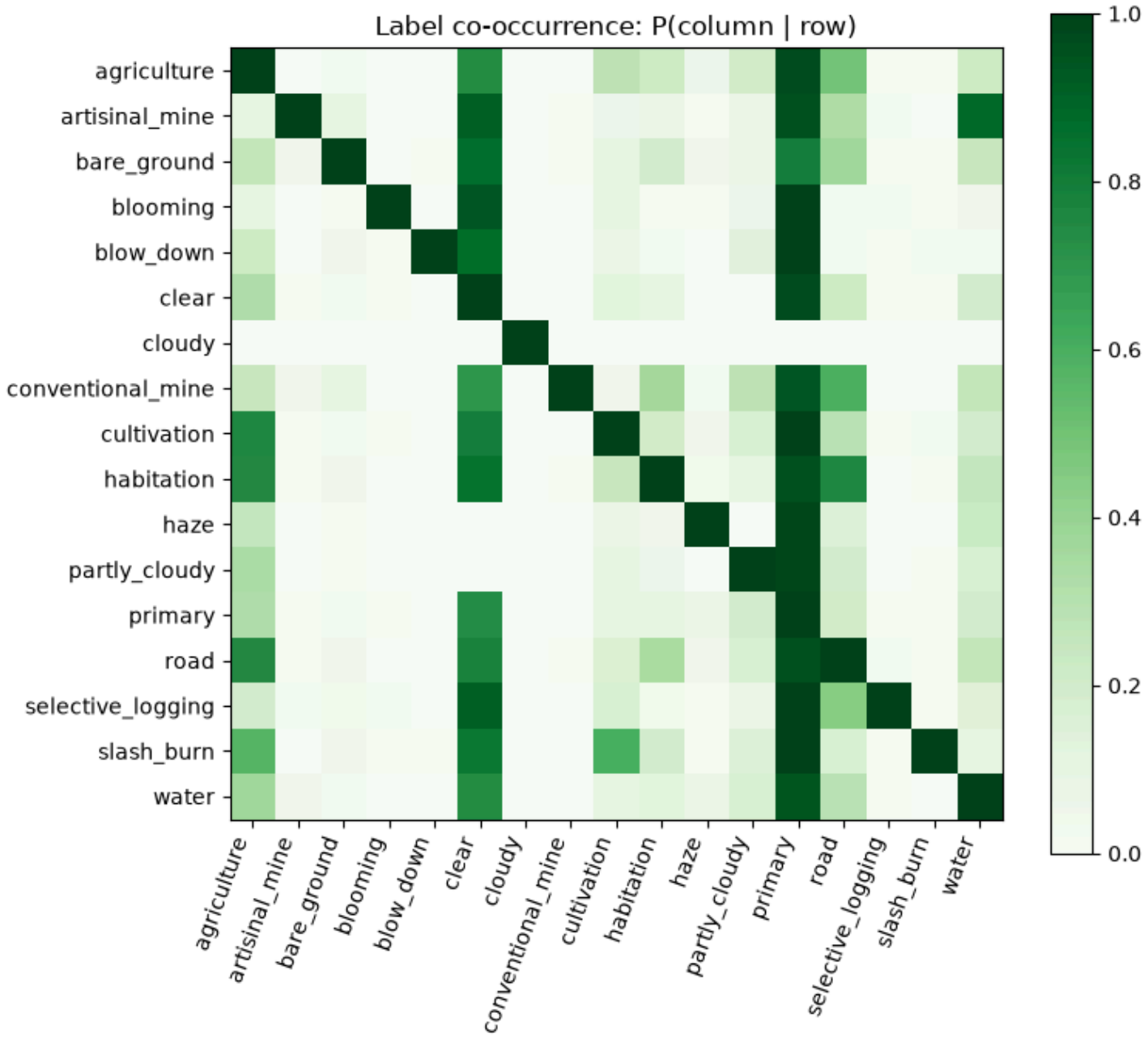
tileSize = 128
subset = None
seed = 42
figDir, cache = f"{root}/figs", f"{root}/cache"
os.makedirs(figDir, exist_ok=True)
os.makedirs(cache, exist_ok=True)
paths = json.load(open(f"{root}/paths.json"))
imageExt = paths.get("image_ext", ".tif").lower()
jpgMode = imageExt in (".jpg", ".jpeg")
```

```
df = pd.read_csv(paths["csv"])
tagLists = df["tags"].str.split()
labels = sorted({t for tl in tagLists for t in tl})
assert len(labels) == 17, labels
y = np.zeros((len(df), 17), dtype=np.uint8)
for i, tl in enumerate(tagLists):
    for t in tl:
        y[i, labels.index(t)] = 1
names = df["image_name"].tolist()
print("Tiles:", len(df), "| labels:", labels)
```

```
Tiles: 40479 | labels: ['agriculture', 'artisinal_mine', 'bare_ground', 'blooming', 'blow_down', 'cle
ar', 'cloudy', 'conventional_mine', 'cultivation', 'habitation', 'haze', 'partly_cloudy', 'primary',
'road', 'selective_logging', 'slash_burn', 'water']
```

```python
counts = pd.Series(y.sum(0), index=labels).sort_values(ascending=False)
table = pd.DataFrame({"tiles": counts, "percent": (100 * counts / len(df)).round(2)})
table.to_csv(f"{figDir}/label_counts.csv")
fig, ax = plt.subplots(figsize=(10, 5))
cols = ["#FF8A3D" if p < 1 else "#3FB6A8" for p in table["percent"]]
ax.bar(table.index, table["tiles"], color=cols)
ax.set_yscale("log")
ax.set_ylabel("Number of tiles (log scale)")
ax.set_title("Label frequency (orange = under 1% of tiles)")
plt.xticks(rotation=60, ha="right")
plt.tight_layout()
plt.savefig(f"{figDir}/fig_label_counts.png", dpi=200)
plt.show()
plt.close()
co = y.T.astype(float) @ y.astype(float)
co = co / np.diag(co)[:, None]
fig, ax = plt.subplots(figsize=(8, 7))
im = ax.imshow(co, cmap="Greens", vmin=0, vmax=1)
ax.set_xticks(range(17))
ax.set_xticklabels(labels, rotation=70, ha="right")
ax.set_yticks(range(17))
ax.set_yticklabels(labels)
ax.set_title("Label co-occurrence: P(column | row)")
plt.colorbar(im)
plt.tight_layout()
plt.savefig(f"{figDir}/fig_label_cooccurrence.png", dpi=200)
plt.show()
plt.close()
```






```
rng = np.random.RandomState(seed)
keep = np.arange(len(df))
if subset and subset < len(df):
    rareIdx = [labels.index(l) for l in labels if counts[l] < 1000]
    isRare = y[:, rareIdx].any(1)
    others = np.where(~isRare)[0]
    need = max(subset - int(isRare.sum()), 0)
    keep = np.sort(np.concatenate([np.where(isRare)[0], rng.choice(others, need, False)]))
y = y[keep]
names = [names[i] for i in keep]
nTiles = len(names)
print("Tiles used:", nTiles)

def image_path(n):
    return os.path.join(paths["tif_dir"], n + imageExt)

def read_image(n):
    """Return float32 HxWxC. TIF: 4 bands; JPG: 3-band RGB (OpenCV BGR->RGB)."""
```

```
    path = image_path(n)
    if jpgMode:
        bgr = cv2.imread(path, cv2.IMREAD_COLOR)
        if bgr is None:
            raise FileNotFoundError(path)
        return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB).astype(np.float32)
    import tifffile
    a = tifffile.imread(path).astype(np.float32)
    if a.ndim == 3 and a.shape[0] == 4 and a.shape[-1] != 4:
        a = np.transpose(a, (1, 2, 0))
    return a
```

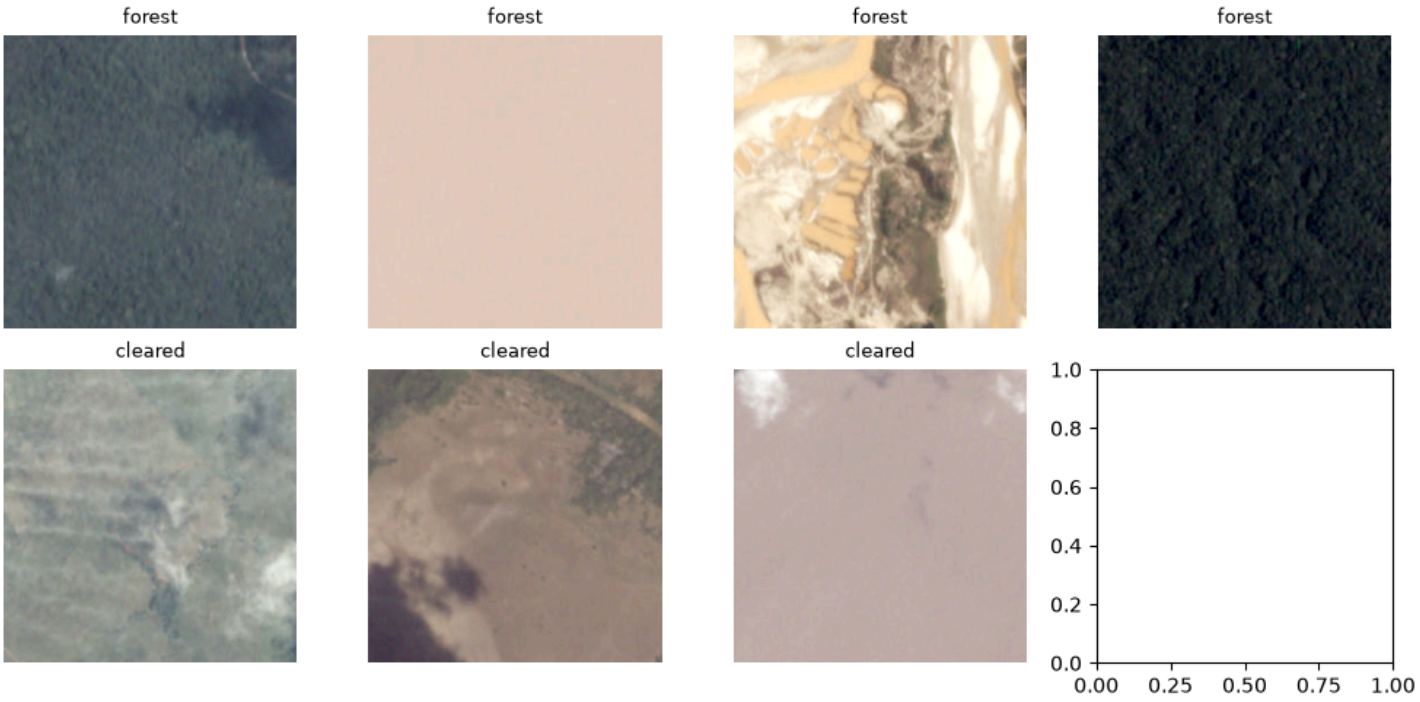
```
Tiles used: 40479
```

```
samp = rng.choice(nTiles, min(600, nTiles), replace=False)
raw = np.stack([read_image(names[i]) for i in samp])
ys = y[samp]
prim = ys[:, labels.index("primary")] == 1
open_ = (ys[:, labels.index("agriculture")] == 1) | (ys[:, labels.index("bare_ground")] == 1)
prim, open_ = prim & ~open_, open_ & ~prim

if jpgMode:
    bandOrder = "RGB_JPG"
    bandMap = dict(r=0, g=1, b=2, nir=None)
    nBands = 3
    tileMean = raw.reshape(len(samp), -1, nBands).mean(1)
    json.dump({"order": bandOrder, "r": 0, "g": 1, "b": 2, "nir": None, "jpg_mode": True},
              open(f"{cache}/order.json", "w"))
    lo = np.percentile(raw.reshape(-1, nBands), 1, axis=0)
    hi = np.percentile(raw.reshape(-1, nBands), 99, axis=0)
    lo5 = np.concatenate([lo, [0.0, 0.0]])
    hi5 = np.concatenate([hi, [1.0, 1.0]])
    json.dump({"lo": lo5.tolist(), "hi": hi5.tolist(), "jpg_mode": True},
              open(f"{cache}/band_stats.json", "w"))
    lo, hi = lo5, hi5
else:
    nBands = 4
    tileMean = raw.reshape(len(samp), -1, 4).mean(1)
    ratio = tileMean[open_].mean(0) / tileMean[prim].mean(0)
    print("Cleared/forest brightness ratio per band index 0..3:", ratio.round(2))
    orders = {"BGRN": dict(b=0, g=1, r=2, nir=3), "RGBN": dict(b=2, g=1, r=0, nir=3)}
    bandOrder = "BGRN"
    if ratio[0] > 1.15 * ratio[2]:
        bandOrder = "RGBN"
    bandMap = orders[bandOrder]
    ndviT = (tileMean[:, bandMap["nir"]] - tileMean[:, bandMap["r"]]) / (
        tileMean[:, bandMap["nir"]] + tileMean[:, bandMap["r"]] + 1e-6)
    for l in ["primary", "agriculture", "water"]:
        m = ys[:, labels.index(l)] == 1
        if m.sum():
            print(f"mean NDVI for {l}: {ndviT[m].mean():.3f}")
    print("Chosen band order:", bandOrder, "-> forest NDVI should be highest, water lowest")
    json.dump({"order": bandOrder, **bandMap, "jpg_mode": False}, open(f"{cache}/order.json", "w"))
    lo = np.percentile(raw.reshape(-1, 4), 1, axis=0)
    hi = np.percentile(raw.reshape(-1, 4), 99, axis=0)
    json.dump({"lo": lo.tolist(), "hi": hi.tolist(), "jpg_mode": False},
              open(f"{cache}/band_stats.json", "w"))

fig, axes = plt.subplots(2, 4, figsize=(10, 5))
```

```
for r, grp in enumerate([np.where(prim)[0][:4], np.where(open_)[0][:4]]):
    for c, k in enumerate(grp):
        if jpgMode:
            img = np.stack([np.clip((raw[k][..., bandMap[x]] - lo[bandMap[x]]) / (hi[bandMap[x]] - lo
                                    0, 1) for x in ("r", "g", "b")], -1) ** 0.8
        else:
            img = np.stack([np.clip((raw[k][..., bandMap[x]] - lo[bandMap[x]]) / (hi[bandMap[x]] - lo
                                    0, 1) for x in ("r", "g", "b")], -1) ** 0.8
        axes[r][c].imshow(img)
        axes[r][c].axis("off")
        axes[r][c].set_title(["forest", "cleared"][r], fontsize=9)
plt.tight_layout()
plt.savefig(f"{figDir}/fig_band_check.png", dpi=150)
plt.show()
plt.close()
del raw
```




```
xmm = np.lib.format.open_memmap(f"{cache}/X.npy", mode="w+", dtype=np.float16,
                                shape=(nTiles, 5, tileSize, tileSize))

def process(i):
    a = read_image(names[i])
    chans = []
    if jpgMode:
        for src in (0, 1, 2):
            chans.append(np.clip((a[..., src] - lo[src]) / (hi[src] - lo[src] + 1e-6), 0, 1))
        chans.append(np.zeros(a.shape[:2], dtype=np.float32))
        chans.append(np.zeros(a.shape[:2], dtype=np.float32))
    else:
        red, nir = a[..., bandMap["r"]], a[..., bandMap["nir"]]
        ndvi = (nir - red) / (nir + red + 1e-6)
        for src in (bandMap["r"], bandMap["g"], bandMap["b"], bandMap["nir"]):
            chans.append(np.clip((a[..., src] - lo[src]) / (hi[src] - lo[src] + 1e-6), 0, 1))
        chans.append((np.clip(ndvi, -1, 1) + 1) / 2)
    arr = np.stack(chans, -1).astype(np.float32)
    arr = cv2.resize(arr, (tileSize, tileSize), interpolation=cv2.INTER_AREA)
```

```
    xmm[i] = np.transpose(arr, (2, 0, 1)).astype(np.float16)

with ThreadPoolExecutor(8) as ex:
    for k, _ in enumerate(ex.map(process, range(nTiles))):
        if k % 2000 == 0:
            print(f"cached {k}/{nTiles}")
xmm.flush()
np.save(f"{cache}/Y.npy", y)
json.dump(labels, open(f"{cache}/labels.json", "w"))
json.dump(names, open(f"{cache}/names.json", "w"))
json.dump({"jpg_mode": jpgMode, "image_ext": imageExt, "channels": ["R", "G", "B", "NIR", "NDVI"],
           "note": "NIR/NDVI are zero placeholders when jpg_mode is true"},
          open(f"{cache}/cache_meta.json", "w"))
```

```
cached 0/40479
cached 2000/40479
cached 4000/40479
cached 6000/40479
cached 8000/40479
cached 10000/40479
cached 12000/40479
cached 14000/40479
cached 16000/40479
cached 18000/40479
cached 20000/40479
cached 22000/40479
cached 24000/40479
cached 26000/40479
cached 28000/40479
cached 30000/40479
cached 32000/40479
cached 34000/40479
cached 36000/40479
cached 38000/40479
cached 40000/40479
```

```
freq = y.sum(0)
strata = np.array([min(np.where(y == 1)[0], key=lambda j: freq[j]) for y in y])
idx = np.arange(nTiles)
tr, tmp = train_test_split(idx, test_size=0.30, stratify=strata, random_state=seed)
va, te = train_test_split(tmp, test_size=0.50, stratify=strata[tmp], random_state=seed)
np.savez(f"{cache}/splits.npz", train=np.sort(tr), val=np.sort(va), test=np.sort(te))
print("train/val/test:", len(tr), len(va), len(te))
print("rare-label counts in test:",
      {l: int(y[te][:, labels.index(l)].sum()) for l in labels if freq[labels.index(l)] < 400})
```

```
train/val/test: 28335 6072 6072
rare-label counts in test: {'artisinal_mine': 51, 'blooming': 51, 'blow_down': 15, 'conventional_min
e': 15, 'selective_logging': 51, 'slash_burn': 31}
```

```
xr = np.load(f"{cache}/X.npy", mmap_mode="r")
pick = np.sort(rng.choice(tr, min(3000, len(tr)), replace=False))
sub = np.asarray(xr[pick]).astype(np.float32)
mean, std = sub.mean((0, 2, 3)), sub.std((0, 2, 3)) + 1e-6
np.savez(f"{cache}/stats.npz", mean=mean, std=std)
```

```
import matplotlib.cm as cm

root = os.environ.get("AMAZON_ROOT", "/mnt/c/Users/legen/amazon_project")
```

```
cache, out = f"{root}/cache", f"{root}/figs/real"
os.makedirs(out, exist_ok=True)
paths = json.load(open(f"{root}/paths.json"))
imageExt = paths.get("image_ext", ".tif").lower()
jpgMode = imageExt in (".jpg", ".jpeg") or json.load(open(f"{cache}/order.json")).get("jpg_mode", Fal
labels = json.load(open(f"{cache}/labels.json"))
names = json.load(open(f"{cache}/names.json"))
y = np.load(f"{cache}/Y.npy")
bandMap = json.load(open(f"{cache}/order.json"))
bandStats = json.load(open(f"{cache}/band_stats.json"))
lo, hi = np.array(bandStats["lo"]), np.array(bandStats["hi"])
rng = np.random.RandomState(7)

def read_image(n):
    path = os.path.join(paths["tif_dir"], n + imageExt)
    if jpgMode:
        bgr = cv2.imread(path, cv2.IMREAD_COLOR)
        if bgr is None:
            raise FileNotFoundError(path)
        return cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB).astype(np.float32)
    import tifffile
    a = tifffile.imread(path).astype(np.float32)
    if a.ndim == 3 and a.shape[0] == 4 and a.shape[-1] != 4:
        a = np.transpose(a, (1, 2, 0))
    return a

def stretch(a, band):
    return np.clip((a[..., band] - lo[band]) / (hi[band] - lo[band] + 1e-6), 0, 1)

def rgb(a, gamma=0.8):
    if jpgMode:
        img = np.stack([stretch(a, 0), stretch(a, 1), stretch(a, 2)], -1)
    else:
        img = np.stack([stretch(a, bandMap["r"]), stretch(a, bandMap["g"]), stretch(a, bandMap["b"])]
    return (255 * img ** gamma).astype(np.uint8)

def save(img, name, size=512):
    img = cv2.resize(img, (size, size), interpolation=cv2.INTER_CUBIC)
    cv2.imwrite(f"{out}/{name}.png", cv2.cvtColor(img, cv2.COLOR_RGB2BGR))

def pick(label, maxTags=None):
    j = labels.index(label)
    cand = np.where(y[:, j] == 1)[0]
    cand = cand[np.argsort(y[cand].sum(1))]
    if maxTags:
        cand = cand[y[cand].sum(1) <= maxTags]
    return int(rng.choice(cand[: max(10, len(cand) // 10)]))
```

```
show = {}
for lab in ["primary", "agriculture", "road", "water", "partly_cloudy", "haze",
            "slash_burn", "artisinal_mine", "selective_logging", "cloudy"]:
    if lab not in labels:
        continue
    i = pick(lab, 3 if lab in ("primary", "water", "haze", "partly_cloudy") else None)
    show[lab] = i
    save(rgb(read_image(names[i])), f"real_chip_{lab}")
```

```
want = [labels.index(l) for l in ["agriculture", "road", "primary"]]
cand = np.where(y[:, want].all(1))[0]
i = int(rng.choice(cand)) if len(cand) else int(rng.randint(0, len(names)))
a = read_image(names[i])
save(rgb(a), "real_rgb")
if jpgMode:
    open(f"{out}/JPG_NOTE.txt", "w").write(
        "CIR and NDVI panels require 4-band TIFs. Only real_rgb was generated from JPGs.\n")
else:
    cir = np.stack([stretch(a, bandMap["nir"]), stretch(a, bandMap["r"]), stretch(a, bandMap["g"])],
    save((255 * cir ** 0.8).astype(np.uint8), "real_cir")
    ndvi = (a[..., bandMap["nir"]] - a[..., bandMap["r"]]) / (a[..., bandMap["nir"]] + a[..., bandMap
    col = cm.RdYlGn((np.clip(ndvi, -0.2, 0.9) + 0.2) / 1.1)[..., :3]
    save((255 * col).astype(np.uint8), "real_ndvi")
```

```
base = rgb(read_image(names[show.get("agriculture", i)]))
variants = [base, np.rot90(base).copy(), base[:, ::-1].copy(),
            np.clip(base[::-1].astype(np.float32) * 1.3, 0, 255).astype(np.uint8)]
for k, v in enumerate(variants):
    save(v, f"real_aug_{k}")
```

```
cand = np.where(y.sum(1) >= 5)[0]
j = int(rng.choice(cand)) if len(cand) else i
save(rgb(read_image(names[j])), "real_multilabel")
```

```
chipSize = 200
prim = np.where(y[:, labels.index("primary")] == 1)[0]
agri = np.where(y[:, labels.index("agriculture")] == 1)[0]
tiles = []
for _ in range(8 * 5):
    pool = agri if rng.rand() < 0.45 else prim
    tiles.append(cv2.resize(rgb(read_image(names[int(rng.choice(pool))])), (chipSize, chipSize)))
rows = [np.concatenate(tiles[r * 8:(r + 1) * 8], 1) for r in range(5)]
mosaic = np.concatenate(rows, 0)[:900]
cv2.imwrite(f"{out}/real_mosaic.png", cv2.cvtColor(mosaic, cv2.COLOR_RGB2BGR))
```

```
True
```

```
import time, random
import torch
import torch.nn as nn
import torchvision
from sklearn.metrics import fbeta_score

root = os.environ.get("AMAZON_ROOT", "/mnt/c/Users/legen/amazon_project")

cache, out = f"{root}/cache", f"{root}/runs"
os.makedirs(out, exist_ok=True)
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Device:", device)
if device != "cuda":
    print("WARNING: no GPU. Training will be extremely slow. Use a CUDA-capable PyTorch build and a G

metaPath = f"{cache}/cache_meta.json"
jpgMode = False
if os.path.exists(metaPath):
    jpgMode = json.load(open(metaPath)).get("jpg_mode", False)
```

```
elif os.path.exists(f"{cache}/order.json"):
    jpgMode = json.load(open(f"{cache}/order.json")).get("jpg_mode", False)

epochs, batch, seed = 14, 128, 42
oversample, tta = True, True
experiments = {
    "A_scratch_rgb":      dict(arch="scratch", chans=[0, 1, 2], lr=3e-3),
    "B_scratch_nir_ndvi": dict(arch="scratch", chans=[0, 1, 2, 3, 4], lr=3e-3),
    "C_resnet_rgb":       dict(arch="resnet", chans=[0, 1, 2], lr=1e-3),
    "D_resnet_nir_ndvi":  dict(arch="resnet", chans=[0, 1, 2, 3, 4], lr=1e-3),
}
skipOnJpg = {"B_scratch_nir_ndvi", "D_resnet_nir_ndvi"}

def seed_all(s):
    random.seed(s)
    np.random.seed(s)
    torch.manual_seed(s)
    torch.cuda.manual_seed_all(s)
```

```
Device: cuda
```

```
xAll = np.load(f"{cache}/X.npy", mmap_mode="r")
y = np.load(f"{cache}/Y.npy")
sp = np.load(f"{cache}/splits.npz")
tr, va, te = sp["train"], sp["val"], sp["test"]
st = np.load(f"{cache}/stats.npz")
chanMean, chanStd = st["mean"], st["std"]
labels = json.load(open(f"{cache}/labels.json"))
nLabels = len(labels)

def load_split(idx, chans, chunk=2000):
    """Copy the chosen channels of the chosen tiles into RAM (float16)."""
    out = torch.empty((len(idx), len(chans), xAll.shape[2], xAll.shape[3]), dtype=torch.float16)
    for s in range(0, len(idx), chunk):
        part = np.asarray(xAll[idx[s:s + chunk]])[:, chans]
        out[s:s + chunk] = torch.from_numpy(part)
    return out
```

```
def block(i, o):
    return nn.Sequential(
        nn.Conv2d(i, o, 3, padding=1, bias=False), nn.BatchNorm2d(o), nn.ReLU(True),
        nn.Conv2d(o, o, 3, padding=1, bias=False), nn.BatchNorm2d(o), nn.ReLU(True),
        nn.MaxPool2d(2))

class ScratchCNN(nn.Module):
    def __init__(self, cIn, nOut):
        super().__init__()
        self.features = nn.Sequential(block(cIn, 32), block(32, 64),
                                      block(64, 128), block(128, 256))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Dropout(0.3), nn.Linear(256, nOut))
    def forward(self, x):
        return self.head(self.features(x))

def build_model(arch, cIn):
    if arch == "scratch":
        return ScratchCNN(cIn, nLabels)
    m = torchvision.models.resnet34(weights=torchvision.models.ResNet34_Weights.IMAGENET1K_V1)
```

```
    old = m.conv1
    new = nn.Conv2d(cIn, 64, 7, 2, 3, bias=False)
    with torch.no_grad():
        new.weight[:, :3] = old.weight
        if cIn > 3:
            extra = old.weight.mean(1, keepdim=True).repeat(1, cIn - 3, 1, 1)
            new.weight[:, 3:] = extra
    m.conv1 = new
    m.fc = nn.Linear(m.fc.in_features, nLabels)
    return m
```

```
def dihedral(x, k):
    if k >= 4:
        x = x.flip(3)
    return torch.rot90(x, k % 4, (2, 3))

def augment(x):
    ks = torch.randint(0, 8, (x.size(0),), device=x.device)
    out = torch.empty_like(x)
    for k in range(8):
        m = ks == k
        if m.any():
            out[m] = dihedral(x[m], k)
    return out

@torch.no_grad()
def predict(model, xs, mean, std, tta=False, bs=256):
    model.eval()
    probs = []
    for s in range(0, len(xs), bs):
        xb = (xs[s:s + bs].to(device).float() - mean) / std
        views = range(8) if tta else [0]
        p = 0
        for k in views:
            with torch.autocast("cuda", dtype=torch.float16, enabled=device == "cuda"):
                p = p + torch.sigmoid(model(dihedral(xb, k)).float())
        probs.append((p / len(views)).cpu().numpy())
    return np.concatenate(probs)

def macro_f2(y, p, thr=0.2):
    return fbeta_score(y, (p >= thr).astype(int), beta=2, average="macro", zero_division=0)
```

```
def run(name, cfg):
    seed_all(seed)
    chans = cfg["chans"]
    mean = torch.tensor(chanMean[chans], device=device).view(1, -1, 1, 1).float()
    std = torch.tensor(chanStd[chans], device=device).view(1, -1, 1, 1).float()
    xtr, xva, xte = (load_split(i, chans) for i in (tr, va, te))
    ytr, yva, yte = y[tr].astype(np.float32), y[va], y[te]
    ytrT = torch.from_numpy(ytr)
    model = build_model(cfg["arch"], len(chans)).to(device)
    freq = ytr.mean(0)
    posW = torch.tensor(np.clip(np.sqrt((1 - freq) / freq), 1, 10), dtype=torch.float32)
    crit = nn.BCEWithLogitsLoss(pos_weight=posW.to(device))
    steps = int(np.ceil(len(tr) / batch))
    opt = torch.optim.AdamW(model.parameters(), lr=cfg["lr"], weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=cfg["lr"], epochs=epochs,
                                                steps_per_epoch=steps)
```

```
    scaler = torch.amp.GradScaler("cuda", enabled=device == "cuda")
    w = (ytr * (freq[None, :] ** -0.5)).max(1)
    w = torch.from_numpy(w / w.sum())
    hist, best, bestState = [], -1.0, None
    for ep in range(epochs):
        model.train()
        t0 = time.time()
        order = torch.multinomial(w, len(tr), replacement=True) if oversample \
            else torch.randperm(len(tr))
        tot = 0.0
        for s in range(0, len(order), batch):
            b = order[s:s + batch]
            xb = (xtr[b].to(device).float() - mean) / std
            yb = ytrT[b].to(device)
            xb = augment(xb)
            opt.zero_grad(set_to_none=True)
            with torch.autocast("cuda", dtype=torch.float16, enabled=device == "cuda"):
                logits = model(xb)
            loss = crit(logits.float(), yb)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            sched.step()
            tot += loss.item() * len(b)
        pv = predict(model, xva, mean, std)
        f2 = macro_f2(yva, pv)
        hist.append(dict(epoch=ep + 1, train_loss=tot / len(tr), val_macro_f2_at_0_2=f2))
        print(f"[{name}] epoch {ep + 1}/{epochs} loss {tot / len(tr):.4f} "
              f"val macroF2@0.2 {f2:.4f} ({time.time() - t0:.0f}s)")
        if f2 > best:
            best = f2
            bestState = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}
    model.load_state_dict(bestState)
    torch.save(bestState, f"{out}/{name}.pt")
    pv, pt = predict(model, xva, mean, std, tta), predict(model, xte, mean, std, tta)
    np.savez_compressed(f"{out}/{name}_probs.npz", val_probs=pv, val_y=yva, test_probs=pt,
                        test_y=yte, val_idx=va, test_idx=te)
    json.dump(hist, open(f"{out}/{name}_history.json", "w"))
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
    ax[0].plot([h["train_loss"] for h in hist])
    ax[0].set_title("Training loss")
    ax[1].plot([h["val_macro_f2_at_0_2"] for h in hist])
    ax[1].set_title("Validation macro F2 (threshold 0.2)")
    for a in ax:
        a.set_xlabel("epoch")
    plt.tight_layout()
    plt.savefig(f"{out}/{name}_curves.png", dpi=160)
    plt.show()
    plt.close()
    print(f"[{name}] best val macro F2@0.2 = {best:.4f}")
    del xtr, xva, xte, model
    if device == "cuda":
        torch.cuda.empty_cache()
```

```
for name, cfg in experiments.items():
    if jpgMode and name in skipOnJpg:
        print("Skipping", name, "- JPG cache has no real NIR/NDVI (need TIFs for B/D).")
        continue
```

```
    if os.path.exists(f"{out}/{name}_probs.npz"):
        print("Skipping", name, "(already done)")
        continue
    run(name, cfg)
```

```
Skipping A_scratch_rgb (already done)
Skipping B_scratch_nir_ndvi - JPG cache has no real NIR/NDVI (need TIFs for B/D).
Skipping C_resnet_rgb (already done)
Skipping D_resnet_nir_ndvi - JPG cache has no real NIR/NDVI (need TIFs for B/D).
```

```
from sklearn.metrics import hamming_loss, recall_score, precision_recall_curve, multilabel_confusion_

root = os.environ.get("AMAZON_ROOT", "/mnt/c/Users/legen/amazon_project")

cache, runs, figDir = f"{root}/cache", f"{root}/runs", f"{root}/figs"
os.makedirs(figDir, exist_ok=True)
os.makedirs(f"{root}/results", exist_ok=True)
labels = json.load(open(f"{cache}/labels.json"))
expNames = ["A_scratch_rgb", "B_scratch_nir_ndvi", "C_resnet_rgb", "D_resnet_nir_ndvi"]
expNames = [n for n in expNames if os.path.exists(f"{runs}/{n}_probs.npz")]
rare = [l for l in ["artisinal_mine", "artisanal_mine", "conventional_mine",
                    "selective_logging", "slash_burn"] if l in labels]
rareIdx = [labels.index(l) for l in rare]
print("Experiments found:", expNames, "| rare labels:", rare)
if not expNames:
    raise SystemExit("No *_probs.npz found in runs/. Train with 04_train_models first.")
```

```
Experiments found: ['A_scratch_rgb', 'C_resnet_rgb'] | rare labels: ['artisinal_mine', 'conventional_
mine', 'selective_logging', 'slash_burn']
```

```
def best_thresholds(y, p, grid=np.arange(0.02, 0.81, 0.01)):
    """Per-label threshold that maximises F2 on the validation set."""
    thr = np.zeros(y.shape[1])
    for j in range(y.shape[1]):
        pred = p[:, j:j + 1] >= grid[None, :]
        pos = y[:, j:j + 1] == 1
        tp = (pred & pos).sum(0)
        fp = (pred & ~pos).sum(0)
        fn = (~pred & pos).sum(0)
        thr[j] = grid[(5 * tp / (5 * tp + 4 * fn + fp + 1e-9)).argmax()]
    return thr

def f2_per_label(y, pred):
    tp = (pred & (y == 1)).sum(0)
    fp = (pred & (y == 0)).sum(0)
    fn = (~pred & (y == 1)).sum(0)
    return 5 * tp / (5 * tp + 4 * fn + fp + 1e-9)

def metrics(y, pred):
    rec = recall_score(y, pred.astype(int), average=None, zero_division=0)
    return {
        "macro_F2": fbeta_score(y, pred, beta=2, average="macro", zero_division=0),
        "samples_F2 (Kaggle metric)": fbeta_score(y, pred, beta=2, average="samples",
                                                  zero_division=0),
        "micro_F2": fbeta_score(y, pred, beta=2, average="micro", zero_division=0),
        "hamming_loss": hamming_loss(y, pred),
        "rare_recall_mean": float(rec[rareIdx].mean()) if rareIdx else float("nan"),
        **{f"recall_{l}": float(rec[labels.index(l)]) for l in rare},
    }
```

```
res, store = {}, {}
for n in expNames:
    d = np.load(f"{runs}/{n}_probs.npz")
    thr = best_thresholds(d["val_y"], d["val_probs"])
    pred = d["test_probs"] >= thr[None, :]
    res[n] = metrics(d["test_y"], pred)
    store[n] = dict(y=d["test_y"], pred=pred, probs=d["test_probs"], thr=thr)
    json.dump(dict(zip(labels, thr.tolist())), open(f"{runs}/thresholds_{n}.json", "w"))
table = pd.DataFrame(res).T
table.to_csv(f"{figDir}/results_table.csv")
print(table.round(4).to_string())
```

```
               macro_F2  samples_F2 (Kaggle metric)  micro_F2  hamming_loss  rare_recall_mean  recall
_artisinal_mine  recall_conventional_mine  recall_selective_logging  recall_slash_burn
A_scratch_rgb    0.6817                     0.9139    0.9015        0.0609            0.6147
0.8235                   0.6000                   0.7451             0.2903
C_resnet_rgb     0.7113                     0.9222    0.9122        0.0529            0.6521
0.9608                   0.6667                   0.5294             0.4516
```

```
rows = ["| Experiment | Macro F2 | Rare-class recall | Hamming loss | Kaggle-style F2 |",
        "|---|---|---|---|---|"]
for n in expNames:
    r = res[n]
    rows.append(f"| {n} | {r['macro_F2']:.3f} | {r['rare_recall_mean']:.3f} | "
                f"{r['hamming_loss']:.4f} | {r['samples_F2 (Kaggle metric)']:.3f} |")
open(f"{figDir}/slide14_table.md", "w").write("\n".join(rows))
```

```
201
```

```
root = os.environ.get("AMAZON_ROOT", "/mnt/c/Users/legen/amazon_project")

rng = np.random.RandomState(0)

def boot_diff(a, b, reps=1000):
    y, pa, pb = store[a]["y"], store[a]["pred"], store[b]["pred"]
    diffs = []
    for _ in range(reps):
        i = rng.randint(0, len(y), len(y))
        diffs.append(f2_per_label(y[i], pb[i]).mean() - f2_per_label(y[i], pa[i]).mean())
    return np.mean(diffs), np.percentile(diffs, 2.5), np.percentile(diffs, 97.5)

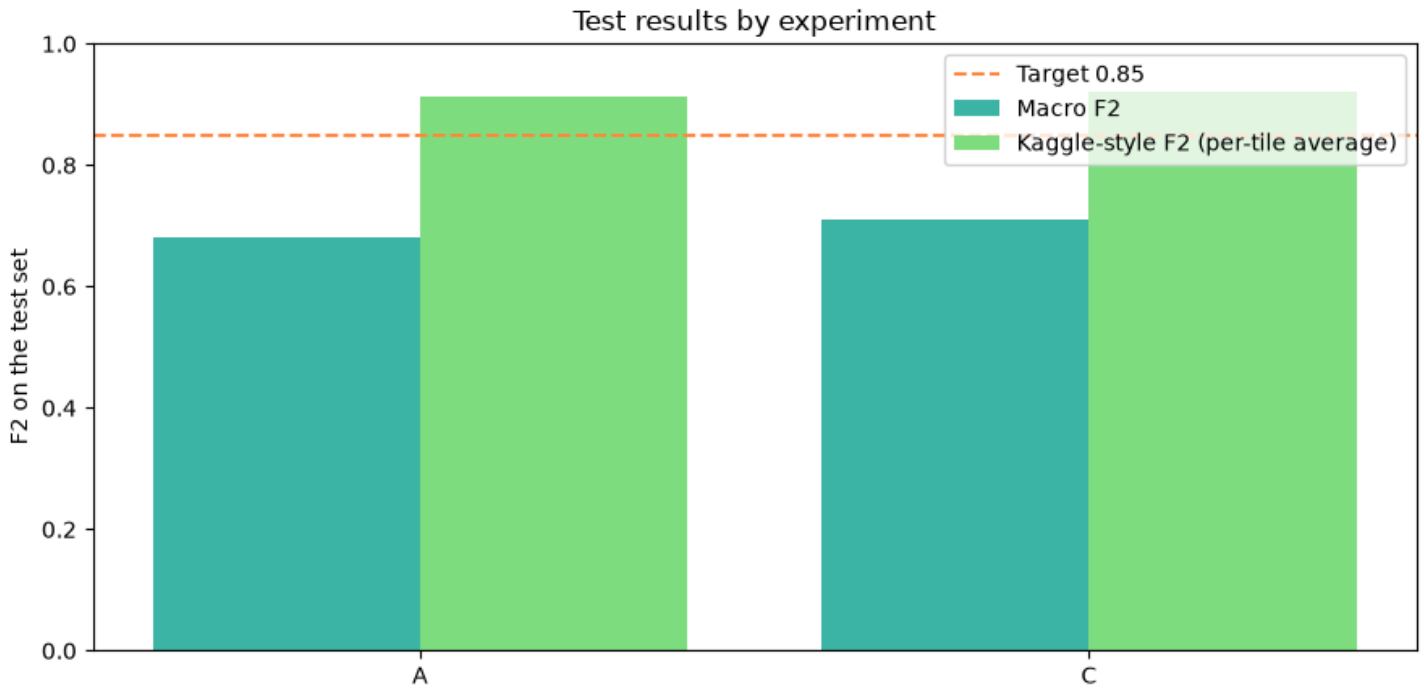
pairs = [
    ("A_scratch_rgb", "B_scratch_nir_ndvi", "NDVI effect, scratch CNN (B - A)"),
    ("C_resnet_rgb", "D_resnet_nir_ndvi", "NDVI effect, ResNet (D - C)"),
    ("A_scratch_rgb", "C_resnet_rgb", "Transfer learning, RGB (C - A)"),
    ("B_scratch_nir_ndvi", "D_resnet_nir_ndvi", "Transfer learning, +NDVI (D - B)"),
]
bootRows = []
for a, b, label in pairs:
    if a in store and b in store:
        m, lo, hi = boot_diff(a, b)
        verdict = "clear gain" if lo > 0 else ("clear loss" if hi < 0 else "not significant")
        bootRows.append(dict(
            comparison=label, mean_macro_F2_diff=m, ci_low=lo, ci_high=hi, verdict=verdict
        ))
boot = pd.DataFrame(bootRows)
if len(boot):
    os.makedirs(f"{root}/results", exist_ok=True)
```

```
    boot.to_csv(f"{root}/results/bootstrap_pairs.csv", index=False)
    boot.to_csv(f"{figDir}/bootstrap_differences.csv", index=False)
    print(boot.to_string(index=False))
    print()
    print(boot.round(4).to_string())
```

```
                  comparison  mean_macro_F2_diff  ci_low  ci_high    verdict
Transfer learning, RGB (C - A)            0.029421  0.0121 0.048478 clear gain

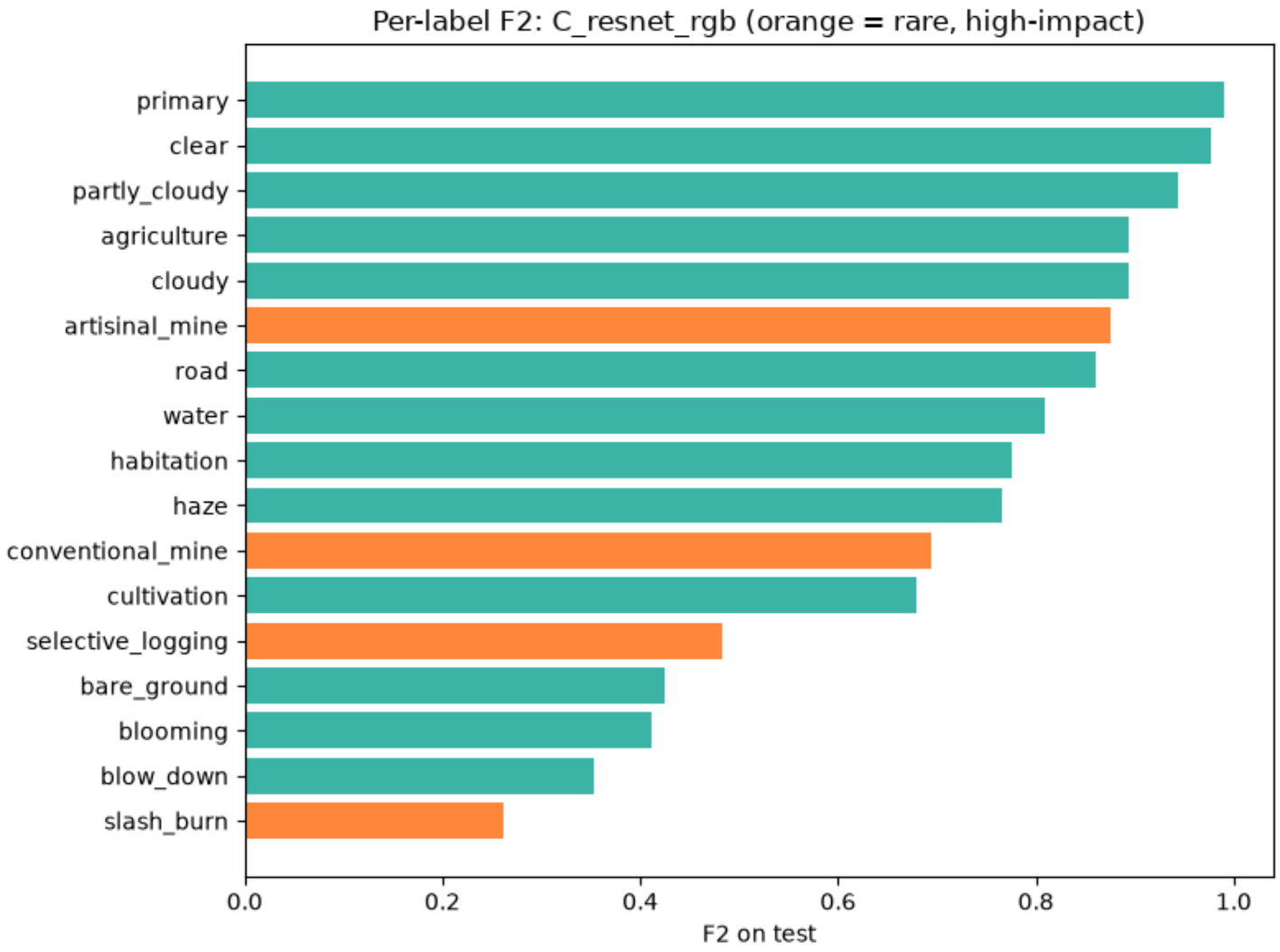
                     comparison  mean_macro_F2_diff  ci_low  ci_high     verdict
0  Transfer learning, RGB (C - A)              0.0294  0.0121   0.0485  clear gain
```

```
fig, ax = plt.subplots(figsize=(9, 4.5))
x = np.arange(len(expNames))
ax.bar(x - 0.2, [res[n]["macro_F2"] for n in expNames], 0.4, label="Macro F2", color="#3FB6A8")
ax.bar(x + 0.2, [res[n]["samples_F2 (Kaggle metric)"] for n in expNames], 0.4,
       label="Kaggle-style F2 (per-tile average)", color="#7EE081")
ax.axhline(0.85, color="#FF8A3D", ls="--", label="Target 0.85")
ax.set_xticks(x)
ax.set_xticklabels([n.split("_", 1)[0] for n in expNames])
ax.set_ylim(0, 1)
ax.set_ylabel("F2 on the test set")
ax.legend()
ax.set_title("Test results by experiment")
plt.tight_layout()
plt.savefig(f"{figDir}/fig_results.png", dpi=200)
plt.show()
plt.close()
```



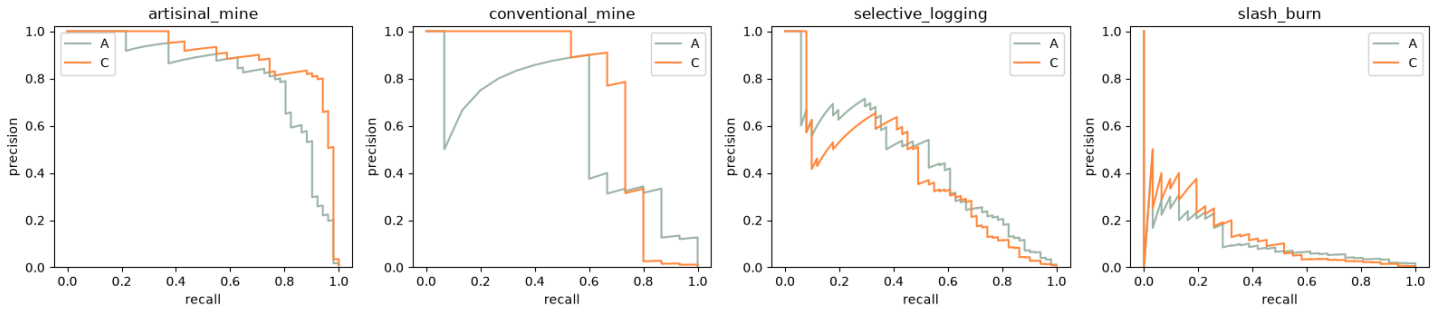

```
best = max(expNames, key=lambda n: res[n]["macro_F2"])
f2l = f2_per_label(store[best]["y"], store[best]["pred"])
order = np.argsort(f2l)
fig, ax = plt.subplots(figsize=(8, 6))
ax.barh([labels[i] for i in order], f2l[order],
        color=["#FF8A3D" if i in rareIdx else "#3FB6A8" for i in order])
ax.set_xlabel("F2 on test")
ax.set_title(f"Per-label F2: {best} (orange = rare, high-impact)")
```

```
plt.tight_layout()
plt.savefig(f"{figDir}/fig_per_label_f2.png", dpi=200)
plt.show()
plt.close()
pd.DataFrame({"label": labels, "F2": f2l}).sort_values("F2").to_csv(
    f"{figDir}/per_label_f2_{best}.csv", index=False)
```

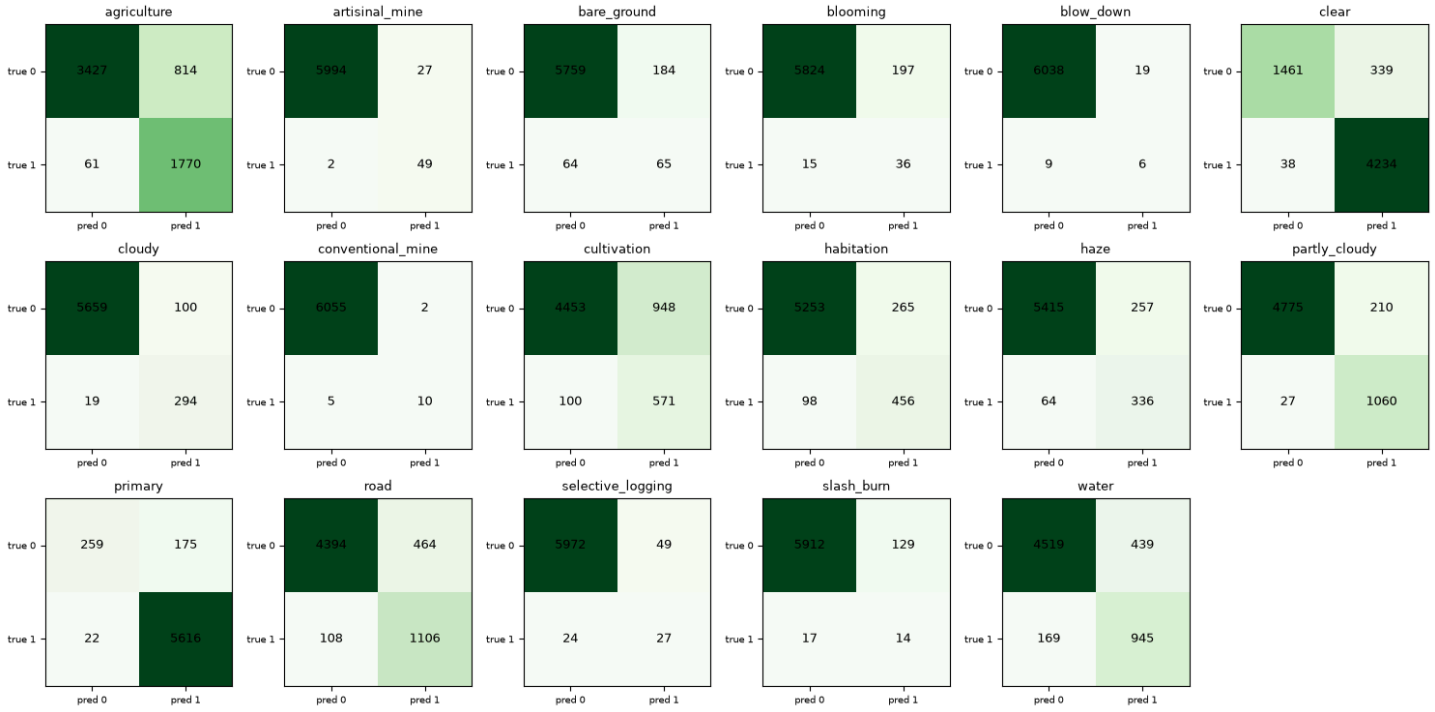



```
if "C_resnet_rgb" in store and "D_resnet_nir_ndvi" in store:
    pair = ("C_resnet_rgb", "D_resnet_nir_ndvi")
elif len(expNames) >= 2:
    pair = (expNames[0], expNames[-1])
else:
    pair = (expNames[0], expNames[0])
if rare:
    fig, axes = plt.subplots(1, len(rare), figsize=(4 * len(rare), 3.6), squeeze=False)
    for ax, l in zip(axes[0], rare):
        j = labels.index(l)
        for n, c in zip(dict.fromkeys(pair), ["#9DB5A6", "#FF8A3D"]):
            p, r, _ = precision_recall_curve(store[n]["y"][:, j], store[n]["probs"][:, j])
            ax.plot(r, p, color=c, label=n.split("_", 1)[0])
        ax.set_title(l)
        ax.set_xlabel("recall")
        ax.set_ylabel("precision")
        ax.set_ylim(0, 1.02)
        ax.legend()
    plt.tight_layout()
```

```
    plt.savefig(f"{figDir}/fig_pr_rare.png", dpi=200)
    plt.show()
    plt.close()
else:
    print("No rare labels found for PR curves.")
```




```
cms = multilabel_confusion_matrix(store[best]["y"], store[best]["pred"].astype(int))
fig, axes = plt.subplots(3, 6, figsize=(15, 7.5))
for ax in axes.ravel():
    ax.axis("off")
for j, ax in enumerate(axes.ravel()[:17]):
    ax.axis("on")
    ax.imshow(cms[j], cmap="Greens")
    for (a, b), v in np.ndenumerate(cms[j]):
        ax.text(b, a, int(v), ha="center", va="center", fontsize=9)
    ax.set_title(labels[j], fontsize=9)
    ax.set_xticks([0, 1])
    ax.set_xticklabels(["pred 0", "pred 1"], fontsize=7)
    ax.set_yticks([0, 1])
    ax.set_yticklabels(["true 0", "true 1"], fontsize=7)
plt.tight_layout()
plt.savefig(f"{figDir}/fig_confusion_matrices.png", dpi=200)
plt.show()
plt.close()
```



```
print("\nBest experiment by macro F2:", best)
for n in expNames:
    r = res[n]
    print(f"{n}: macro F2 {r['macro_F2']:.3f} | Kaggle-style F2 "
          f"{r['samples_F2 (Kaggle metric)']:.3f} | rare recall {r['rare_recall_mean']:.3f}")
print("Targets: macro F2 >= 0.85, rare recall >= 0.60. Report honestly if you miss them.")
print("Figures saved in", figDir)
```

```
Best experiment by macro F2: C_resnet_rgb
A_scratch_rgb: macro F2 0.682 | Kaggle-style F2 0.914 | rare recall 0.615
C_resnet_rgb: macro F2 0.711 | Kaggle-style F2 0.922 | rare recall 0.652
Targets: macro F2 >= 0.85, rare recall >= 0.60. Report honestly if you miss them.
Figures saved in /mnt/c/Users/legen/amazon_project/figs
```

```
import torch.nn.functional as F

root = os.environ.get("AMAZON_ROOT", "/mnt/c/Users/legen/amazon_project")

cache, runs, figDir = f"{root}/cache", f"{root}/runs", f"{root}/figs"
os.makedirs(figDir, exist_ok=True)
device = "cuda" if torch.cuda.is_available() else "cpu"
labels = json.load(open(f"{cache}/labels.json"))
nLabels = len(labels)
xAll = np.load(f"{cache}/X.npy", mmap_mode="r")
st = np.load(f"{cache}/stats.npz")
chanMean, chanStd = st["mean"], st["std"]
expChans = {"C_resnet_rgb": [0, 1, 2], "D_resnet_nir_ndvi": [0, 1, 2, 3, 4]}
expChans = {n: ch for n, ch in expChans.items() if os.path.exists(f"{runs}/{n}.pt")}
targets = [l for l in ["selective_logging", "slash_burn", "artisinal_mine", "agriculture"]
           if l in labels]
print("Models available for Grad-CAM:", list(expChans.keys()))
if not expChans:
    raise SystemExit("No ResNet checkpoints found. Train C (and D if TIFs) in 04 first.")
```

```
Models available for Grad-CAM: ['C_resnet_rgb']
```

```
import torchvision

def build_resnet(cIn):
    m = torchvision.models.resnet34(weights=None)
    m.conv1 = nn.Conv2d(cIn, 64, 7, 2, 3, bias=False)
    m.fc = nn.Linear(m.fc.in_features, nLabels)
    return m

models, data = {}, {}
for n, ch in expChans.items():
    m = build_resnet(len(ch))
    try:
        state = torch.load(f"{runs}/{n}.pt", map_location="cpu", weights_only=True)
    except TypeError:
        state = torch.load(f"{runs}/{n}.pt", map_location="cpu")
    m.load_state_dict(state)
    models[n] = m.to(device).eval()
    d = np.load(f"{runs}/{n}_probs.npz")
    thr = np.array([json.load(open(f"{runs}/thresholds_{n}.json"))[l] for l in labels])
    data[n] = dict(pred=d["test_probs"] >= thr[None, :], y=d["test_y"], idx=d["test_idx"])

ref = "C_resnet_rgb" if "C_resnet_rgb" in data else next(iter(data))
testIdx = data[ref]["idx"]
```

```
y = data[ref]["y"]
if "C_resnet_rgb" in data and "D_resnet_nir_ndvi" in data:
    assert (data["C_resnet_rgb"]["idx"] == data["D_resnet_nir_ndvi"]["idx"]).all()
```

```
def gradcam(model, x, labelJ):
    acts, grads = [], []
    def save_act(mod, inp, out):
        acts.append(out)
        out.register_hook(lambda g: grads.append(g))
    h = model.layer4.register_forward_hook(save_act)
    model.zero_grad()
    out = model(x)
    out[0, labelJ].backward()
    h.remove()
    w = grads[0].mean((2, 3), keepdim=True)
    cam = F.relu((w * acts[0]).sum(1, keepdim=True))
    cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)[0, 0]
    cam = cam - cam.min()
    return (cam / (cam.max() + 1e-8)).detach().cpu().numpy()

def prep(gi, n):
    ch = expChans[n]
    x = torch.from_numpy(np.asarray(xAll[gi])[ch].astype(np.float32))[None].to(device)
    mean = torch.tensor(chanMean[ch], device=device).view(1, -1, 1, 1).float()
    std = torch.tensor(chanStd[ch], device=device).view(1, -1, 1, 1).float()
    return (x - mean) / std

def rgb_view(gi):
    return np.clip(np.transpose(np.asarray(xAll[gi])[:3].astype(np.float32), (1, 2, 0)) ** 0.8, 0, 1)
```

```
hasBoth = "C_resnet_rgb" in data and "D_resnet_nir_ndvi" in data
for lab in targets:
    j = labels.index(lab)
    truth = y[:, j] == 1
    if hasBoth:
        cData, dData = data["C_resnet_rgb"], data["D_resnet_nir_ndvi"]
        fixed = np.where(truth & ~cData["pred"][:, j] & dData["pred"][:, j])[0][:3]
        both = np.where(truth & ~cData["pred"][:, j] & ~dData["pred"][:, j])[0][:2]
        picks = [("D fixes C", i) for i in fixed] + [("both miss", i) for i in both]
        modelNames = ["C_resnet_rgb", "D_resnet_nir_ndvi"]
        titles = ["RGB-only model", "NIR+NDVI model"]
        ncols = 3
    else:
        n0 = ref
        miss = np.where(truth & ~data[n0]["pred"][:, j])[0][:3]
        hit = np.where(truth & data[n0]["pred"][:, j])[0][:2]
        picks = [("model miss", i) for i in miss] + [("model hit", i) for i in hit]
        modelNames = [n0]
        titles = ["RGB Grad-CAM"]
        ncols = 2
    if not picks:
        continue
    fig, axes = plt.subplots(len(picks), ncols, figsize=(3 * ncols, 2.7 * len(picks)),
                             squeeze=False)
    for r, (kind, i) in enumerate(picks):
        gi = int(testIdx[i])
        base = rgb_view(gi)
        axes[r][0].imshow(base)
```

```
        axes[r][0].set_title(f"{kind} ({lab})", fontsize=8)
        for c, n in enumerate(modelNames, start=1):
            cam = gradcam(models[n], prep(gi, n), j)
            axes[r][c].imshow(base)
            axes[r][c].imshow(cam, cmap="jet", alpha=0.45)
            axes[r][c].set_title(titles[c - 1], fontsize=8)
        for a in axes[r]:
            a.axis("off")
    plt.tight_layout()
    plt.savefig(f"{figDir}/fig_gradcam_{lab}.png", dpi=200)
    plt.show()
    plt.close()
```

model miss (selective_logging)



RGB Grad-CAM



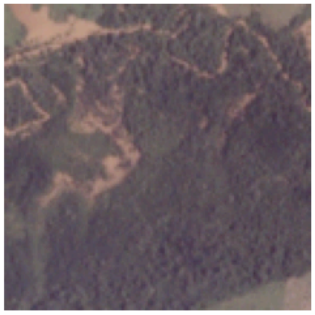
model miss (selective_logging)



RGB Grad-CAM



model miss (selective_logging)



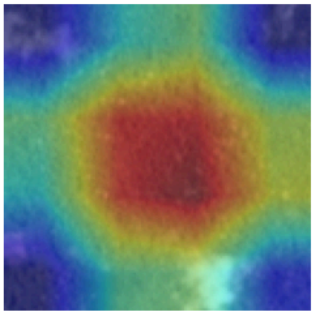
RGB Grad-CAM



model hit (selective_logging)



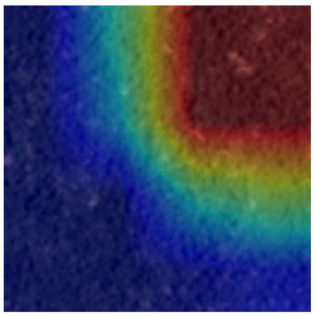
RGB Grad-CAM



model hit (selective_logging)



RGB Grad-CAM



model miss (slash_burn)



RGB Grad-CAM



model miss (slash_burn)



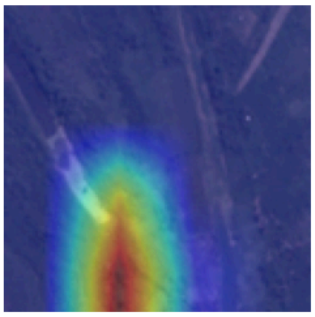
RGB Grad-CAM



model miss (slash_burn)



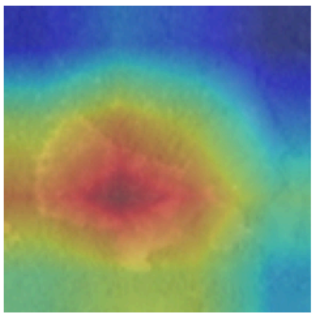
RGB Grad-CAM



model hit (slash_burn)



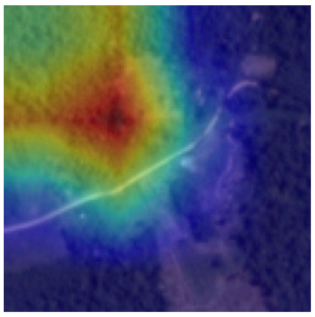
RGB Grad-CAM



model hit (slash_burn)



RGB Grad-CAM



model miss (artisinal_mine)



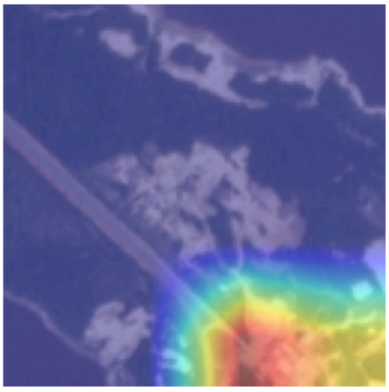
RGB Grad-CAM



model miss (artisinal_mine)



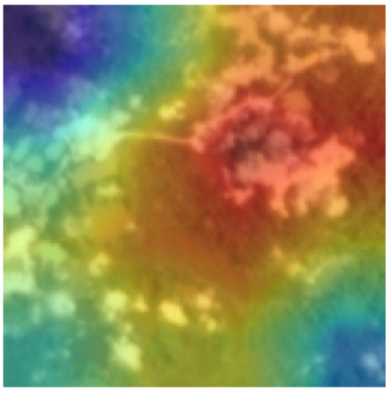
RGB Grad-CAM



model hit (artisinal_mine)



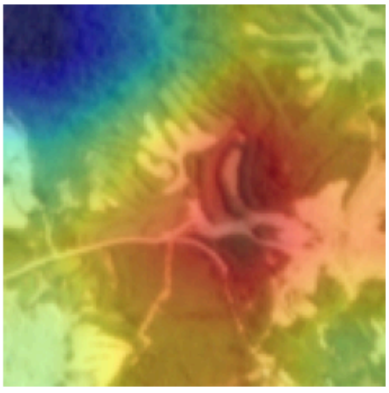
RGB Grad-CAM

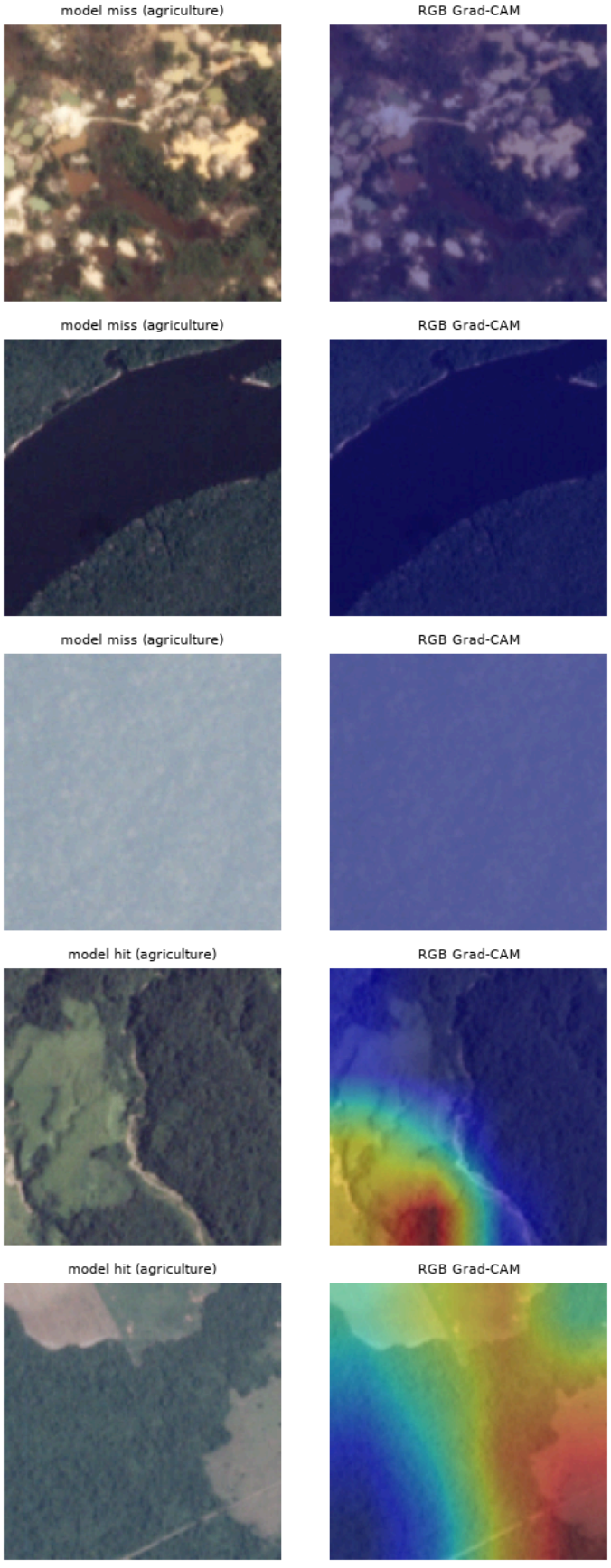


model hit (artisinal_mine)



RGB Grad-CAM






```
weather = [w for w in ["clear", "partly_cloudy", "haze", "cloudy"] if w in labels]
rows = []
for n in expChans:
    wrong = (data[n]["pred"].astype(int) != y).mean(1)
    for w in weather:
```

```
        m = y[:, labels.index(w)] == 1
        rows.append(dict(model=n, weather=w, tiles=int(m.sum()),
                         mean_label_error=float(wrong[m].mean())))
err = pd.DataFrame(rows)
err.to_csv(f"{figDir}/error_by_weather.csv", index=False)
pv = err.pivot(index="weather", columns="model", values="mean_label_error").loc[weather]
colors = ["#9DB5A6", "#FF8A3D"][: len(pv.columns)]
pv.plot.bar(figsize=(7, 4), color=colors)
plt.ylabel("share of the 17 labels predicted wrong")
plt.title("Error by weather condition (test set)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(f"{figDir}/fig_error_by_weather.png", dpi=200)
plt.show()
plt.close()
```

